Kuruba Eswar Prem Sai Ranjith

Day 2 – Set 1: E-Commerce Catalogue and Traffic Analysis

Task 1: Binary Search Tree of product categories

```
class Node:
    def __init__(self, name):
        self.name = name
        self.left = None
        self.right = None

def insert(root, name):
    if root is None:
        return Node(name)
    if name.lower() < root.name.lower():
        root.left = insert(root.left, name)
    elif name.lower() > root.name.lower():
        root.right = insert(root.right, name)
    return root

def search(root, name):
    steps = 0
    while root:
        steps += 1
        if name.lower() == root.name.lower():
            return True, steps
        root = root.left if name.lower() < root.name.lower() else root.right
    return False, steps

def inorder(r):
    return inorder(r.left) + [r.name] + inorder(r.right) if r else []

def preorder(r):
    return [r.name] + preorder(r.left) + preorder(r.right) if r else []

def postorder(r):
    return postorder(r.left) + postorder(r.right) + [r.name] if r else []

cats = ["Mobiles", "Laptops", "Books", "Footwear", "Toys", "Grocery", "Kitchen",
        "Furniture", "Beauty", "Sports", "Watches", "Cameras", "Fashion",
        "Stationery", "Appliances"]

root = None
for c in cats:
    root = insert(root, c)

print("Inorder  :", inorder(root))
print("Preorder :", preorder(root))
print("Postorder:", postorder(root))
for name in ["Cameras", "Toys", "Garden"]:
    found, steps = search(root, name)
    print("Search", name, "->", found, "in", steps, "steps")
```

Output

```
Inorder  : ['Appliances', 'Beauty', 'Books', 'Cameras', 'Fashion', 'Footwear', 'Furniture',
'Grocery', 'Kitchen', 'Laptops', 'Mobiles', 'Sports', 'Stationery', 'Toys', 'Watches']
Preorder : ['Mobiles', 'Laptops', 'Books', 'Beauty', 'Appliances', 'Footwear', 'Cameras', 'Fashion',
'Grocery', 'Furniture', 'Kitchen', 'Toys', 'Sports', 'Stationery', 'Watches']
Postorder: ['Appliances', 'Beauty', 'Fashion', 'Cameras', 'Furniture', 'Kitchen', 'Grocery',
'Footwear', 'Books', 'Laptops', 'Stationery', 'Sports', 'Watches', 'Toys', 'Mobiles']
Search Cameras -> True in 5 steps
Search Toys -> True in 2 steps
Search Garden -> False in 6 steps
```

Task 2: SKU look-up with a hash table (chaining) and the built-in dict

```
import time, random

class HashTable:
    def __init__(self, size):
        self.size = size
        self.buckets = [[] for _ in range(size)]

    def hash(self, key):
        h = 0
        for ch in key:
            h = (h * 31 + ord(ch)) % self.size
        return h

    def put(self, key, value):
        bucket = self.buckets[self.hash(key)]
        for item in bucket:
            if item[0] == key:
                item[1] = value
                return
        bucket.append([key, value])

    def get(self, key):
        for k, v in self.buckets[self.hash(key)]:
            if k == key:
                return v
        return None

# Part A: collision demo with a tiny table (7 SKUs, 5 buckets)
small = HashTable(5)
items = {"SKU101": "Earbuds", "SKU102": "Power bank", "SKU103": "Mouse",
         "SKU104": "Keyboard", "SKU105": "Webcam", "SKU106": "Router", "SKU107": "Hub"}
for k, v in items.items():
    small.put(k, v)
    print(k, "-> bucket", small.hash(k))
print("Buckets:")
for i, b in enumerate(small.buckets):
    print(" ", i, [k for k, _ in b])
print("get(SKU106) =", small.get("SKU106"), "| get(SKU999) =", small.get("SKU999"))

# Part B: 100,000 SKUs, own table vs built-in dict
N = 100000
skus = ["SKU" + str(100000 + i) for i in range(N)]
random.seed(7)
random.shuffle(skus)

mine = HashTable(100003)
for s in skus:
    mine.put(s, len(s))
builtin = {s: len(s) for s in skus}

t = time.perf_counter()
for s in skus:
    mine.get(s)
t_mine = time.perf_counter() - t

t = time.perf_counter()
for s in skus:
    builtin.get(s)
t_dict = time.perf_counter() - t

longest = max(len(b) for b in mine.buckets)
print("\nLookups for", N, "SKUs")
print("Own hash table :", round(t_mine, 4), "s  (longest chain:", longest, ")")
print("Built-in dict  :", round(t_dict, 4), "s")
print("dict is about", round(t_mine / t_dict, 1), "times faster")
```

Output

```
SKU101 -> bucket 4
SKU102 -> bucket 0
```

```
SKU103 -> bucket 1
SKU104 -> bucket 2
SKU105 -> bucket 3
SKU106 -> bucket 4
SKU107 -> bucket 0
Buckets:
  0 ['SKU102', 'SKU107']
  1 ['SKU103']
  2 ['SKU104']
  3 ['SKU105']
  4 ['SKU101', 'SKU106']
get(SKU106) = Router | get(SKU999) = None

Lookups for 100000 SKUs
Own hash table : 0.1219 s  (longest chain: 4 )
Built-in dict  : 0.0184 s
dict is about 6.6 times faster
```

Task 3: Microservice deployment order and circular dependency (DFS and BFS)

```
from collections import deque

# "A depends on B" means B must be deployed before A
depends = [
    ("gateway", "cart"), ("gateway", "catalog"), ("gateway", "search"),
    ("cart", "pricing"), ("pricing", "inventory"), ("inventory", "db"),
    ("orders", "cart"), ("orders", "payments"), ("payments", "users"),
    ("payments", "db"), ("notify", "orders"), ("catalog", "db"),
    ("catalog", "cache"), ("search", "catalog"), ("users", "db"),
]
services = sorted({s for pair in depends for s in pair})

def build(edges):
    needs = {s: [] for s in services}
    for a, b in edges:
        needs[a].append(b)
    return needs

def find_cycle(needs):
    state = {s: 0 for s in needs}
    path = []
    def dfs(u):
        state[u] = 1
        path.append(u)
        for v in needs[u]:
            if state[v] == 1:
                return path[path.index(v):] + [v]
            if state[v] == 0:
                c = dfs(v)
                if c:
                    return c
        path.pop()
        state[u] = 2
        return None
    for s in needs:
        if state[s] == 0:
            c = dfs(s)
            if c:
                return c
    return None

def dfs_order(needs):
    seen, order = set(), []
    def dfs(u):
        seen.add(u)
        for v in needs[u]:
            if v not in seen:
                dfs(v)
        order.append(u)
    for s in needs:
```

```
        if s not in seen:
            dfs(s)
    return order

def bfs_order(needs):
    waiting = {s: len(needs[s]) for s in needs}
    users = {s: [] for s in needs}
    for s in needs:
        for d in needs[s]:
            users[d].append(s)
    q = deque(s for s in needs if waiting[s] == 0)
    order = []
    while q:
        s = q.popleft()
        order.append(s)
        for u in users[s]:
            waiting[u] -= 1
            if waiting[u] == 0:
                q.append(u)
    return order

needs = build(depends)
print("Cycle in original graph:", find_cycle(needs))
print("DFS deployment order:", dfs_order(needs))
print("BFS deployment order:", bfs_order(needs))

bad = build(depends + [("db", "notify")])      # db now needs notify
print("\nAfter adding db -> notify")
print("Cycle found:", " -> ".join(find_cycle(bad)))
print("BFS can only place", len(bfs_order(bad)), "of", len(services), "services")
```

Output

```
Cycle in original graph: None
DFS deployment order: ['cache', 'db', 'inventory', 'pricing', 'cart', 'catalog', 'search', 'gateway',
'users', 'payments', 'orders', 'notify']
BFS deployment order: ['cache', 'db', 'catalog', 'inventory', 'users', 'search', 'pricing',
'payments', 'cart', 'gateway', 'orders', 'notify']

After adding db -> notify
Cycle found: cart -> pricing -> inventory -> db -> notify -> orders -> cart
BFS can only place 1 of 12 services
```

Task 4: Busiest continuous 3-hour window (sliding window)

```
# average requests per minute for each hour of the day (hour 0 to 23)
rpm = [210, 180, 150, 140, 160, 220, 340, 520, 690, 810, 880, 940,
       990, 930, 870, 900, 1010, 1180, 1320, 1410, 1290, 980, 620, 380]

K = 3
window = sum(rpm[:K])
best, start = window, 0
for i in range(K, len(rpm)):
    window += rpm[i] - rpm[i - K]
    if window > best:
        best, start = window, i - K + 1

print("Busiest", K, "hour window:", str(start) + ":00 to " + str(start + K) + ":00")
print("Hours in window:", rpm[start:start + K])
print("Total (rpm added up):", best, "| average rpm:", round(best / K, 1))
```

Output

```
Busiest 3 hour window: 18:00 to 21:00
Hours in window: [1320, 1410, 1290]
Total (rpm added up): 4020 | average rpm: 1340.0
```

Task 5: Gift-card pairs (two pointers)

```
orders = [150, 300, 400, 400, 550, 700, 800, 950, 1100, 1100, 1250, 1400, 1600, 1850]
gift = 1500

left, right = 0, len(orders) - 1
pairs = []
while left < right:
    s = orders[left] + orders[right]
    if s == gift:
        pairs.append((orders[left], orders[right]))
        l, r = orders[left], orders[right]
        while left < right and orders[left] == l:
            left += 1
        while left < right and orders[right] == r:
            right -= 1
    elif s < gift:
        left += 1
    else:
        right -= 1

print("Sorted order values:", orders)
print("Gift card value:", gift)
print("Pairs:", pairs)
```

Output

```
Sorted order values: [150, 300, 400, 400, 550, 700, 800, 950, 1100, 1100, 1250, 1400, 1600, 1850]
Gift card value: 1500
Pairs: [(400, 1100), (550, 950), (700, 800)]
```

Decision table

Task	Problem	Data structure / pattern	Time / space	Reason
1	Browse categories in order and find one quickly	Binary Search Tree with inorder, preorder and postorder	O(log n) balanced, O(n) worst; space O(n)	Smaller names go left and larger go right, so each step skips part of the tree. Inorder gives the alphabetical list with no extra sort.
2	Instant look-up among 100,000 SKUs	Hash table with chaining, compared with dict	O(1) average; space O(n)	The hash picks the bucket directly. Colliding SKUs share a chain, so none are lost. The built-in dict does the same job in C and was faster in my run.
3	Valid deployment order and circular dependency	Directed graph with DFS and BFS (in-degree counting)	O(V + E); space O(V + E)	Dependencies are edges. DFS puts a service after everything it needs and shows the exact cycle. BFS places services whose dependencies are done and stalls when there is a cycle.
4	Busiest 3-hour window in 24 hours	Sliding window (fixed size 3)	O(n); space O(1)	Neighbouring windows share 2 of 3 hours, so each step adds one hour and removes one instead of adding all three again.
5	Pairs of orders that add up to the gift-card value	Two pointers on the sorted list	O(n); space O(1) apart from the answer	The list is sorted, so a sum that is too small moves the left pointer up and a sum that is too big moves the right pointer down. One pass is enough.

Day 2 – Set 2: Insurance Claims Processing

Task 1: Binary Search Tree of claim IDs

Pseudocode

```
START
  Node has: cid, left, right

  INSERT(root, cid):
    IF root is empty: RETURN new Node(cid)
    IF cid < root.cid: root.left = INSERT(root.left, cid)
    ELSE IF cid > root.cid: root.right = INSERT(root.right, cid)
    RETURN root                    (equal id is ignored, no duplicate stored)

  SEARCH(root, cid): walk down left or right until found or the tree ends

  INORDER = left, node, right | PREORDER = node, left, right | POSTORDER = left, right, node

  INSERT all claim IDs (one of them twice)
  PRINT the three traversals and check that INORDER is sorted
  SEARCH two IDs, one present and one missing
END
```

Code

```
class Node:
    def __init__(self, cid):
        self.cid = cid
        self.left = None
        self.right = None

def insert(root, cid):
    if root is None:
        return Node(cid)
    if cid < root.cid:
        root.left = insert(root.left, cid)
    elif cid > root.cid:
        root.right = insert(root.right, cid)
    return root                    # equal id: ignored, no duplicate stored

def search(root, cid):
    while root:
        if cid == root.cid:
            return True
        root = root.left if cid < root.cid else root.right
    return False

def inorder(r):
    return inorder(r.left) + [r.cid] + inorder(r.right) if r else []

def preorder(r):
    return [r.cid] + preorder(r.left) + preorder(r.right) if r else []

def postorder(r):
    return postorder(r.left) + postorder(r.right) + [r.cid] if r else []

if __name__ == "__main__":
    ids = [7450, 7210, 7800, 7105, 7300, 7650, 7920, 7150, 7380, 7700, 7210, 7990]
    root = None
    for i in ids:
        root = insert(root, i)
    print("Claim IDs entered :", ids)
    print("Inorder   :", inorder(root))
    print("Preorder  :", preorder(root))
    print("Postorder :", postorder(root))
    print("Inorder is sorted:", inorder(root) == sorted(set(ids)))
    for q in [7380, 7400]:
        print("Search", q, "->", search(root, q))
```

Output

```
Claim IDs entered : [7450, 7210, 7800, 7105, 7300, 7650, 7920, 7150, 7380, 7700, 7210, 7990]
Inorder   : [7105, 7150, 7210, 7300, 7380, 7450, 7650, 7700, 7800, 7920, 7990]
Preorder  : [7450, 7210, 7105, 7150, 7300, 7380, 7800, 7650, 7700, 7920, 7990]
Postorder : [7150, 7105, 7380, 7300, 7210, 7700, 7650, 7990, 7920, 7800, 7450]
Inorder is sorted: True
Search 7380 -> True
Search 7400 -> False
```

Task 2: Duplicate and near-duplicate claims (hashing)

Pseudocode

```
START
  NORMALISE(policy, date):
    remove all spaces, change policy to upper case
    RETURN policy + "|" + date                      (this is the key)

  ClaimTable has: size, buckets (list of empty lists), collisions = 0

  HASH(key): h = (h * 31 + code of ch) MOD size for every character

  ADD(claim_id, policy, date):
    key = NORMALISE(policy, date)
    bucket = buckets[HASH(key)]
    FOR each (stored_key, stored_id) in bucket:
      IF stored_key == key: RETURN duplicate of stored_id
    IF bucket is not empty: collisions = collisions + 1     (different key, same bucket)
    add (key, claim_id) to the bucket
    RETURN not a duplicate

  FOR each claim: ADD it and collect the duplicates
  PRINT duplicates, number of collisions and the bucket contents
END
```

Code

```
def normalise(policy, date):
    return policy.replace(" ", "").upper() + "|" + date.replace(" ", "")

class ClaimTable:
    def __init__(self, size=4):
        self.size = size
        self.buckets = [[] for _ in range(size)]
        self.collisions = 0

    def hash(self, key):
        h = 0
        for ch in key:
            h = (h * 31 + ord(ch)) % self.size
        return h

    def add(self, claim_id, policy, date):
        key = normalise(policy, date)
        bucket = self.buckets[self.hash(key)]
        for k, cid in bucket:
            if k == key:
                return True, cid              # same key already stored
        if bucket:
            self.collisions += 1              # different key, same bucket
        bucket.append((key, claim_id))
        return False, claim_id

def find_duplicates(claims, size=4):
    table = ClaimTable(size)
    dups = []
    for cid, policy, date in claims:
        is_dup, first = table.add(cid, policy, date)
        if is_dup:
```

```
            dups.append((cid, first))
    return dups, table

if __name__ == "__main__":
    claims = [
        ("C101", "PL-5521", "2026-10-03"),
        ("C102", "PL-7788", "2026-10-03"),
        ("C103", "pl-5521", "2026-10-03"),
        ("C104", "PL-3109", "2026-10-05"),
        ("C105", "PL -7788", "2026-10-03"),
        ("C106", "PL-5521", "2026-10-09"),
        ("C107", "Pl-3109 ", "2026-10-05"),
        ("C108", "PL-4402", "2026-10-06"),
        ("C109", "PL-4402", "2026-10-06"),
        ("C110", "PL-9014", "2026-10-07"),
    ]
    dups, table = find_duplicates(claims, 4)
    print("Claims checked:", len(claims), "| duplicates found:", len(dups))
    for later, first in dups:
        print(" ", later, "is a duplicate of", first)
    print("Hash collisions handled by chaining:", table.collisions)
    for i, b in enumerate(table.buckets):
        print(" bucket", i, [k for k, _ in b])
```

Output

```
Claims checked: 10 | duplicates found: 4
  C103 is a duplicate of C101
  C105 is a duplicate of C102
  C107 is a duplicate of C104
  C109 is a duplicate of C108
Hash collisions handled by chaining: 2
 bucket 0 ['PL-3109|2026-10-05', 'PL-5521|2026-10-09']
 bucket 1 ['PL-7788|2026-10-03']
 bucket 2 ['PL-5521|2026-10-03', 'PL-4402|2026-10-06']
 bucket 3 ['PL-9014|2026-10-07']
```

Task 3: Claim approval stages as a directed graph (BFS and DFS)

Pseudocode

```
START
  graph[stage] = list of stages it can move to

  SHORTEST_PATH(start, goal)               (BFS)
    parent[start] = none, queue = [start]
    WHILE queue is not empty:
      u = remove first item of queue
      IF u == goal: follow parent back to start and RETURN the path reversed
      FOR each v in graph[u]:
        IF v has no parent yet: parent[v] = u, add v to queue
    RETURN no path

  FIND_LOOPS()                             (DFS)
    mark every stage NEW
    DFS(u): mark u ON-PATH, push u on path
      FOR each v in graph[u]:
        IF v is ON-PATH: record loop = path from v to u, then v
        ELSE IF v is NEW: DFS(v)
      pop u, mark u DONE

  PRINT the graph, three shortest paths and every loop found
END
```

Code

```
from collections import deque

graph = {
    "Registration": ["Assessment"],
```

```
    "Assessment":   ["Surveyor", "Settlement"],
    "Surveyor":     ["Settlement", "Assessment"],
    "Settlement":   ["Registration"],       # claim reopened
}

def shortest_path(g, start, goal):
    parent = {start: None}
    q = deque([start])
    while q:
        u = q.popleft()
        if u == goal:
            path = []
            while u is not None:
                path.append(u)
                u = parent[u]
            return path[::-1]
        for v in g[u]:
            if v not in parent:
                parent[v] = u
                q.append(v)
    return None

def find_loops(g):
    state = {n: 0 for n in g}          # 0 new, 1 on current path, 2 done
    path, loops = [], []
    def dfs(u):
        state[u] = 1
        path.append(u)
        for v in g[u]:
            if state[v] == 1:
                loops.append(path[path.index(v):] + [v])
            elif state[v] == 0:
                dfs(v)
        path.pop()
        state[u] = 2
    for n in g:
        if state[n] == 0:
            dfs(n)
    return loops

if __name__ == "__main__":
    for n, nxt in graph.items():
        print(n, "->", nxt)
    print()
    for a, b in [("Registration", "Settlement"), ("Surveyor", "Registration"),
                 ("Assessment", "Assessment")]:
        p = shortest_path(graph, a, b)
        print("Shortest", a, "to", b, ":", " > ".join(p), "(", len(p) - 1, "steps )")
    print()
    for loop in find_loops(graph):
        print("Loop:", " > ".join(loop))
```

Output

```
Registration -> ['Assessment']
Assessment -> ['Surveyor', 'Settlement']
Surveyor -> ['Settlement', 'Assessment']
Settlement -> ['Registration']

Shortest Registration to Settlement : Registration > Assessment > Settlement ( 2 steps )
Shortest Surveyor to Registration : Surveyor > Settlement > Registration ( 2 steps )
Shortest Assessment to Assessment : Assessment ( 0 steps )

Loop: Registration > Assessment > Surveyor > Settlement > Registration
Loop: Assessment > Surveyor > Assessment
```

Task 4: Longest run of days above 100 claims (sliding window)

Pseudocode

```
START
  LONGEST_RUN(claims, limit = 100):
    best = 0, best_start = -1, left = 0
    FOR right from 0 to last day:
      IF claims[right] <= limit: left = right + 1       (window broken, restart after it)
      ELSE IF window length (right - left + 1) > best:
        best = window length, best_start = left
    RETURN best, best_start
  PRINT the longest run, its first and last day and its values
END
```

Code

```
def longest_run(claims, limit=100):
    best, best_start, left = 0, -1, 0
    for right in range(len(claims)):
        if claims[right] <= limit:        # day breaks the window
            left = right + 1
        elif right - left + 1 > best:
            best = right - left + 1
            best_start = left
    return best, best_start

if __name__ == "__main__":
    days = [72, 85, 91, 104, 118, 96, 88, 79, 101, 99, 66, 94, 83, 87, 112,
            135, 158, 171, 149, 126, 108, 142, 100, 93, 89, 105, 117, 110, 92, 84]
    length, start = longest_run(days)
    print("Claims per day (day 1 to 30):", days)
    print("Longest run above 100:", length, "days")
    print("From day", start + 1, "to day", start + length)
    print("Values:", days[start:start + length])
```

Output

```
Claims per day (day 1 to 30): [72, 85, 91, 104, 118, 96, 88, 79, 101, 99, 66, 94, 83, 87, 112, 135,
158, 171, 149, 126, 108, 142, 100, 93, 89, 105, 117, 110, 92, 84]
Longest run above 100: 8 days
From day 15 to day 22
Values: [112, 135, 158, 171, 149, 126, 108, 142]
```

Task 5: Merging two sorted regional lists (two pointers)

Pseudocode

```
START
  MERGE(a, b):
    i = 0, j = 0, out = empty list
    WHILE i < length of a AND j < length of b:
      IF a[i] <= b[j]: add a[i] to out, i = i + 1
      ELSE: add b[j] to out, j = j + 1
    add the rest of a (from i) to out
    add the rest of b (from j) to out
    RETURN out
  PRINT both office lists, the merged list, and check it equals sorted(a + b)
END
```

Code

```
def merge(a, b):
    i = j = 0
    out = []
    while i < len(a) and j < len(b):
        if a[i] <= b[j]:
            out.append(a[i])
```

```
            i += 1
        else:
            out.append(b[j])
            j += 1
    out.extend(a[i:])                    # leftover of whichever list is not finished
    out.extend(b[j:])
    return out

if __name__ == "__main__":
    mumbai = [1002, 1015, 1015, 1040, 1066, 1090, 1123]
    kolkata = [1001, 1015, 1038, 1066, 1075, 1150, 1180, 1201]
    merged = merge(mumbai, kolkata)
    print("Mumbai office :", mumbai)
    print("Kolkata office:", kolkata)
    print("Merged        :", merged)
    print("Same as sorted(a + b):", merged == sorted(mumbai + kolkata))
```

Output

```
Mumbai office : [1002, 1015, 1015, 1040, 1066, 1090, 1123]
Kolkata office: [1001, 1015, 1038, 1066, 1075, 1150, 1180, 1201]
Merged        : [1001, 1002, 1015, 1015, 1015, 1038, 1040, 1066, 1066, 1075, 1090, 1123, 1150, 1180,
1201]
Same as sorted(a + b): True
```

Test cases (empty input and duplicates)

Pseudocode

```
START
  CHECK(name, got, expected): PRINT PASS or FAIL for the test and stop on a FAIL
  FOR each task 1 to 5:
    run it on an empty input and CHECK the result
    run it on duplicate input and CHECK the result
    run a few other cases (loops, missing path, exactly 100, unchanged input lists)
  PRINT "All tests passed" if nothing failed
END
```

Code

```
from task1 import insert, search, inorder, preorder, postorder
from task2 import find_duplicates
from task3 import shortest_path, find_loops
from task4 import longest_run
from task5 import merge

def check(name, got, expected):
    status = "PASS" if got == expected else "FAIL"
    print(status, "-", name)
    assert got == expected

# Task 1: BST
check("T1 empty tree inorder", inorder(None), [])
check("T1 empty tree search", search(None, 5), False)
r = None
for x in [5, 5, 5]:
    r = insert(r, x)
check("T1 same id three times keeps one node", inorder(r), [5])
r = None
for x in [50, 30, 70, 20]:
    r = insert(r, x)
check("T1 preorder", preorder(r), [50, 30, 20, 70])
check("T1 postorder", postorder(r), [20, 30, 70, 50])

# Task 2: hashing
check("T2 empty claim list", find_duplicates([])[0], [])
check("T2 case + space duplicate",
      find_duplicates([("A", "pl-1", "d1"), ("B", " PL-1", "d1")])[0], [("B", "A")])
check("T2 same policy, different date is not duplicate",
```

```
      find_duplicates([("A", "PL-1", "d1"), ("B", "PL-1", "d2")])[0], [])
check("T2 one bucket still correct",
      find_duplicates([("A", "PL-1", "d1"), ("B", "PL-2", "d1"), ("C", "pl-1", "d1")], 1)[0],
      [("C", "A")])

# Task 3: graph
g = {"A": ["B"], "B": ["C"], "C": []}
check("T3 shortest path", shortest_path(g, "A", "C"), ["A", "B", "C"])
check("T3 no path", shortest_path(g, "C", "A"), None)
check("T3 no loops", find_loops(g), [])
check("T3 loop found", find_loops({"A": ["B"], "B": ["A"]}), [["A", "B", "A"]])
check("T3 empty graph", find_loops({}), [])

# Task 4: sliding window
check("T4 empty list", longest_run([]), (0, -1))
check("T4 exactly 100 does not count", longest_run([100, 100]), (0, -1))
check("T4 duplicate values", longest_run([101, 101, 101]), (3, 0))
check("T4 run at the end", longest_run([50, 120, 130]), (2, 1))

# Task 5: merge
check("T5 both empty", merge([], []), [])
check("T5 one empty", merge([], [1, 2]), [1, 2])
check("T5 duplicates kept", merge([1, 1, 3], [1, 3]), [1, 1, 1, 3, 3])
check("T5 input lists unchanged", (lambda a, b: (merge(a, b), a, b)[1:])([1, 4], [2]), ([1, 4], [2]))

print("\nAll tests passed")
```

Output

```
PASS - T1 empty tree inorder
PASS - T1 empty tree search
PASS - T1 same id three times keeps one node
PASS - T1 preorder
PASS - T1 postorder
PASS - T2 empty claim list
PASS - T2 case + space duplicate
PASS - T2 same policy, different date is not duplicate
PASS - T2 one bucket still correct
PASS - T3 shortest path
PASS - T3 no path
PASS - T3 no loops
PASS - T3 loop found
PASS - T3 empty graph
PASS - T4 empty list
PASS - T4 exactly 100 does not count
PASS - T4 duplicate values
PASS - T4 run at the end
PASS - T5 both empty
PASS - T5 one empty
PASS - T5 duplicates kept
PASS - T5 input lists unchanged

All tests passed
```

Complexity table

Task	Solution	Data structure / pattern	Time	Space
1	Insert, search, three traversals	Binary Search Tree	Insert and search O(h): O(log n) balanced, O(n) for sorted input. Traversals O(n)	O(n)
2	Duplicate claim detection	Hash table with chaining on a cleaned key	O(1) average per claim, O(n) worst case	O(n)
3	Shortest stage path and loops	Directed graph, BFS and DFS	O(V + E)	O(V)
4	Longest run above 100	Sliding window (variable size)	O(n)	O(1)

5	Merge two sorted lists	Two pointers	O(m + n)	O(m + n) for the merged list

Day 2 – Set 3: Logistics Network and Delivery Windows

Task 1: BST of hub codes and the effect of sorted input

Pseudocode

```
START
  Node has: code, left, right

  INSERT(root, code): same as a normal BST insert (code changed to upper case first)
  SEARCH(root, code): walk down and count steps
  HEIGHT(root): IF root is empty RETURN 0 ELSE RETURN 1 + larger of HEIGHT(left), HEIGHT(right)

  build the tree from the given opening order, PRINT its height
  SEARCH three hubs
  FOR each order (given, A to Z, Z to A): build the tree and PRINT its height
END
```

Code

```
class Node:
    def __init__(self, code):
        self.code = code
        self.left = None
        self.right = None

def insert(root, code):
    code = code.upper()
    if root is None:
        return Node(code)
    if code < root.code:
        root.left = insert(root.left, code)
    elif code > root.code:
        root.right = insert(root.right, code)
    return root

def search(root, code):
    code = code.upper()
    steps = 0
    while root:
        steps += 1
        if code == root.code:
            return True, steps
        root = root.left if code < root.code else root.right
    return False, steps

def height(root):
    return 0 if root is None else 1 + max(height(root.left), height(root.right))

def build(order):
    root = None
    for c in order:
        root = insert(root, c)
    return root

hubs = ["BPL", "DEL", "BOM", "MAA", "BLR", "HYD", "CCU", "LKO", "PNQ", "VTZ"]

given = build(hubs)
print("Given insertion order:", hubs)
print("Height (levels):", height(given))
for h in ["LKO", "VTZ", "GOI"]:
    print("Search", h, "->", search(given, h))

print("\nSame 10 hubs, different insertion orders")
for name, order in [("given order", hubs),
                    ("sorted A to Z", sorted(hubs)),
```

```
                    ("sorted Z to A", sorted(hubs, reverse=True))]:
    print(" ", name.ljust(14), "height =", height(build(order)))
```

Output

```
Given insertion order: ['BPL', 'DEL', 'BOM', 'MAA', 'BLR', 'HYD', 'CCU', 'LKO', 'PNQ', 'VTZ']
Height (levels): 5
Search LKO -> (True, 5)
Search VTZ -> (True, 5)
Search GOI -> (False, 4)

Same 10 hubs, different insertion orders
  given order    height = 5
  sorted A to Z  height = 10
  sorted Z to A  height = 10
```

Sorted input: every new code is larger than all codes already in the tree, so it always goes to the right and the tree becomes a straight line (height 10 for 10 hubs). Searches then cost up to n steps instead of about log n.

Task 2: Parcel tracking look-up and double scans at the same hub (hash map)

Pseudocode

```
START
  lookup = empty map          (tracking number -> list of (hub, time))
  seen   = empty map          ((tracking number, hub) -> list of times)

  FOR each scan (tracking, hub, time):
    add (hub, time) to lookup[tracking]
    add time to seen[(tracking, hub)]

  PRINT lookup of one existing and one missing tracking number
  FOR each key in seen:
    IF it has more than one time: PRINT the parcel, the hub and the scan times
END
```

Code

```
scans = [
    ("TRK501", "BPL", "08:10"), ("TRK501", "DEL", "15:40"), ("TRK501", "DEL", "15:52"),
    ("TRK502", "BOM", "09:05"), ("TRK502", "BPL", "13:30"), ("TRK502", "HYD", "20:15"),
    ("TRK503", "CCU", "10:00"), ("TRK503", "CCU", "10:03"), ("TRK503", "CCU", "10:07"),
    ("TRK504", "MAA", "11:45"),
    ("TRK505", "BLR", "07:20"), ("TRK505", "HYD", "12:10"), ("TRK505", "BLR", "18:30"),
    ("TRK506", "LKO", "09:50"), ("TRK506", "BPL", "14:05"),
]

# hash map 1: tracking number -> list of (hub, time)
lookup = {}
for trk, hub, t in scans:
    lookup.setdefault(trk, []).append((hub, t))

# hash map 2: (tracking number, hub) -> list of scan times
seen = {}
for trk, hub, t in scans:
    seen.setdefault((trk, hub), []).append(t)

print("Parcels loaded:", len(lookup), "| total scans:", len(scans))
for trk in ["TRK502", "TRK999"]:
    print("Lookup", trk, "->", lookup.get(trk, "not found"))

print("\nScanned more than once at the same hub:")
for (trk, hub), times in seen.items():
    if len(times) > 1:
        print(" ", trk, "at", hub, "->", len(times), "scans at", times)
```

Output

```
Parcels loaded: 6 | total scans: 15
Lookup TRK502 -> [('BOM', '09:05'), ('BPL', '13:30'), ('HYD', '20:15')]
Lookup TRK999 -> not found

Scanned more than once at the same hub:
  TRK501 at DEL -> 2 scans at ['15:40', '15:52']
  TRK503 at CCU -> 3 scans at ['10:00', '10:03', '10:07']
  TRK505 at BLR -> 2 scans at ['07:20', '18:30']
```

Task 3: Hub network as an adjacency list (BFS and DFS)

Pseudocode

```
START
  adj = every hub with an empty list
  FOR each route (a, b): add b to adj[a] and a to adj[b]      (routes go both ways)
  sort every list so the visiting order is the same each run

  FEWEST_HOPS(start, goal)                 (BFS)
    parent[start] = none, queue = [start]
    WHILE queue is not empty:
      u = remove first item of queue
      IF u == goal: follow parent back to start and RETURN the path reversed
      FOR each v in adj[u]: IF v has no parent yet: parent[v] = u, add v to queue
    RETURN no route

  REACHABLE(start)                         (DFS)
    DFS(u): mark u visited, add u to order
            FOR each v in adj[u]: IF v is not visited: DFS(v)
    RETURN order

  PRINT the adjacency list, four BFS routes, and the hubs reachable from the central depot
END
```

Code

```
from collections import deque

hubs = ["BPL", "DEL", "BOM", "MAA", "BLR", "HYD", "CCU", "LKO", "GAU", "IMF"]
routes = [("BPL", "DEL"), ("BPL", "BOM"), ("BPL", "HYD"), ("BPL", "LKO"),
          ("DEL", "LKO"), ("DEL", "CCU"), ("BOM", "MAA"), ("HYD", "BLR"),
          ("BLR", "MAA"), ("CCU", "MAA"), ("GAU", "IMF")]
depot = "BPL"

adj = {h: [] for h in hubs}
for a, b in routes:
    adj[a].append(b)
    adj[b].append(a)
for h in adj:
    adj[h].sort()

def fewest_hops(start, goal):
    parent = {start: None}
    q = deque([start])
    while q:
        u = q.popleft()
        if u == goal:
            path = []
            while u is not None:
                path.append(u)
                u = parent[u]
            return path[::-1]
        for v in adj[u]:
            if v not in parent:
                parent[v] = u
                q.append(v)
    return None
```

```
def reachable(start):
    seen, order = set(), []
    def dfs(u):
        seen.add(u)
        order.append(u)
        for v in adj[u]:
            if v not in seen:
                dfs(v)
    dfs(start)
    return order

print("Adjacency list")
for h in hubs:
    print(" ", h, "->", adj[h])

print("\nFewest hops (BFS)")
for a, b in [("LKO", "MAA"), ("BLR", "CCU"), ("DEL", "HYD"), ("BOM", "GAU")]:
    p = fewest_hops(a, b)
    print(" ", a, "to", b, ":", " > ".join(p) + " (" + str(len(p) - 1) + " hops)" if p else "no
route")

r = reachable(depot)
print("\nReachable from", depot, "(DFS):", r)
print("Count:", len(r), "of", len(hubs), "| not reachable:", sorted(set(hubs) - set(r)))
```

Output

```
Adjacency list
  BPL -> ['BOM', 'DEL', 'HYD', 'LKO']
  DEL -> ['BPL', 'CCU', 'LKO']
  BOM -> ['BPL', 'MAA']
  MAA -> ['BLR', 'BOM', 'CCU']
  BLR -> ['HYD', 'MAA']
  HYD -> ['BLR', 'BPL']
  CCU -> ['DEL', 'MAA']
  LKO -> ['BPL', 'DEL']
  GAU -> ['IMF']
  IMF -> ['GAU']

Fewest hops (BFS)
  LKO to MAA : LKO > BPL > BOM > MAA (3 hops)
  BLR to CCU : BLR > MAA > CCU (2 hops)
  DEL to HYD : DEL > BPL > HYD (2 hops)
  BOM to GAU : no route

Reachable from BPL (DFS): ['BPL', 'BOM', 'MAA', 'BLR', 'HYD', 'CCU', 'DEL', 'LKO']
Count: 8 of 10 | not reachable: ['GAU', 'IMF']
```

Task 4: Highest-average 4-hour window (sliding window)

Pseudocode

```
START
  parcels = 24 hourly counts, K = 4
  window = sum of the first K hours
  best = window, start = 0
  FOR i from K to 23:
    window = window + parcels[i] - parcels[i - K]
    IF window > best: best = window, start = i - K + 1
  PRINT the window, its hours, total and average (best / K)
END
```

Code

```
# parcels handled in each hour of the day (hour 0 to 23)
parcels = [120, 95, 80, 70, 110, 240, 480, 760, 920, 840, 710, 650,
           600, 580, 620, 700, 880, 1050, 1180, 990, 760, 540, 330, 190]

K = 4
```

```
window = sum(parcels[:K])
best, start = window, 0
for i in range(K, len(parcels)):
    window += parcels[i] - parcels[i - K]
    if window > best:
        best, start = window, i - K + 1

print("Highest average", K, "hour window:", str(start) + ":00 to " + str(start + K) + ":00")
print("Hours in window:", parcels[start:start + K])
print("Total:", best, "| average per hour:", best / K)
```

Output

```
Highest average 4 hour window: 16:00 to 20:00
Hours in window: [880, 1050, 1180, 990]
Total: 4100 | average per hour: 1025.0
```

Task 5: Route palindrome of hub codes (two pointers)

Pseudocode

```
START
  IS_ROUTE_PALINDROME(text):
    hubs = all letter/digit groups in text, changed to upper case   (separators dropped)
    left = 0, right = last index of hubs
    WHILE left < right:
      IF hubs[left] != hubs[right]: RETURN False
      left = left + 1, right = right - 1
    RETURN True
  PRINT the result for each sample route description
END
```

Code

```
import re

def is_route_palindrome(text):
    hubs = [h.upper() for h in re.findall(r"[A-Za-z0-9]+", text)]
    left, right = 0, len(hubs) - 1
    while left < right:
        if hubs[left] != hubs[right]:
            return False
        left += 1
        right -= 1
    return True

routes = [
    "BPL-DEL-BOM-DEL-BPL",
    "bpl > del > lko > del > BPL",
    "MAA / BLR / maa",
    "HYD, BPL, HYD",
    "BPL-DEL-BOM",
    "DEL-BPL-DEL-BPL",
    "BLR - MAA - CCU - MAA - BLR -",
    "BPL",
    "",
]
for r in routes:
    print(repr(r).ljust(34), is_route_palindrome(r))
```

Output

```
'BPL-DEL-BOM-DEL-BPL'              True
'bpl > del > lko > del > BPL'      True
'MAA / BLR / maa'                  True
'HYD, BPL, HYD'                    True
'BPL-DEL-BOM'                      False
'DEL-BPL-DEL-BPL'                  False
```

```
'BLR - MAA - CCU - MAA - BLR -'    True
'BPL'                              True
''                                 True
```

Algorithm choices

Task	Algorithm	Why I chose it	Time / space
1	Binary Search Tree	Hub codes have an order, so a BST gives a sorted list and fast search. It must stay balanced, because sorted insertion turns it into a line.	O(h) per insert or search; space O(n)
2	Hash map (dict)	The question is whether this exact (tracking number, hub) pair has been seen before, which a hash map answers in constant time during a single pass. Comparing every pair of scans would be O(n squared).	O(n) time and space
3	Adjacency list with BFS and DFS	The network is sparse, so a list is smaller than a matrix. Routes have no weights, so BFS gives the fewest hops. DFS visits every hub that can be reached from the depot.	O(V + E); space O(V)
4	Sliding window (size 4)	With a fixed window length the highest total is also the highest average. Each step adds one hour and removes one.	O(n); space O(1)
5	Two pointers	Compares the first hub with the last and moves inwards, stopping at the first mismatch. No reversed copy is needed.	O(n) time; space O(n) for the list of codes